

Service-Oriented Architecture Case Study

Online Shopping System

The Online Shopping System case study is a highly distributed World Wide Web-based system that provides services for purchasing items such as books or clothes. The solution uses a service-oriented architecture with multiple services; coordinator objects are used to facilitate the integration of the services. In addition, object brokers are used to provide service registration, brokering, and discovery. Services include a catalog service, an inventory service, a customer account service, a delivery order service, an email service, and a credit card authorization service.

The problem is described in Section 22.1. Section 22.2 describes the use case model for the Online Shopping System. Section 22.3 describes the static model, which includes the system context model that depicts the boundary between the system and the external environment. This section also describes the use of broker technology in this system before going on to describe static modeling of the entity classes. Section 22.4 describes how to structure the system into objects. Section 22.5 describes dynamic modeling, in which communication diagrams are developed for each of the use cases. Section 22.6 describes the design model for the system, which is designed as a layered architecture based on the Layers of Abstraction pattern consisting of services and components.

22.1 PROBLEM DESCRIPTION

In the Web-based Online Shopping System, customers can request to purchase one or more items from the supplier. The customer provides personal details, such as address and credit card information. This information is stored in a customer account. If the credit card is valid, then a delivery order is created and sent to the supplier. The supplier checks the available inventory, confirms the order, and enters a planned shipping date. When the order is shipped, the customer is notified and the customer's credit card account is charged.

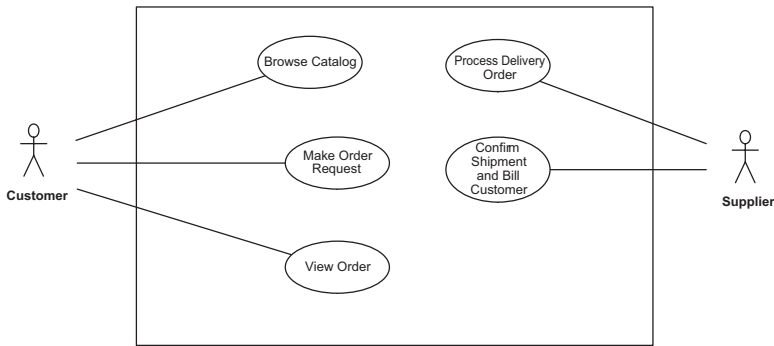


Figure 22.1. Web-based Online Shopping System: use cases

22.2 USE CASE MODELING

The use case model for Web-based Online Shopping System is depicted in Figure 22.1. There are two actors: Customer, who browses a catalog and requests to purchase items, and Supplier, who provides the catalog and services customer purchase requests. The customer initiates three use cases, which are Browse Catalog, to browse the catalog and select items; Make Order Request, to make a purchase request; and View Order. The supplier initiates two use cases, namely, Process Delivery Order, to service the customer's order, and Confirm Shipment and Bill Customer, to finalize the purchase.

In the Browse Catalog use case, the customer browses a World Wide Web catalog, views various catalog items from a given supplier's catalog, and selects items from the catalog. In the Make Purchase Request use case, the customer enters personal details. The system creates a customer account if one does not already exist. The customer's credit card is checked for validity and sufficient credit to pay for the requested catalog items. If the credit card check shows that the credit card is valid and has sufficient credit, then the customer purchase is approved and the system sends the purchase request to the supplier. In View Order, the customer requests to view the details of the delivery order.

The supplier-initiated use cases are Process Delivery Order and Confirm Shipment and Bill Customer. In the Process Delivery Order use case, the supplier requests a delivery order, determines that the inventory is available to fulfill the order, and displays the order.

In the Confirm Shipment and Bill Customer use case, the supplier prepares the shipment manually and confirms that the shipment is ready for shipment. The system then retrieves the customer's credit card details from the customer account and bills the customer's credit card.

The use cases are described in detail, except for the very simple View Order use case. Each use case is described textually and is depicted using an activity diagram. Activity diagrams are popular in business process modeling. They can be integrated into the analysis and design of service-oriented applications to model the sequence of use case activities. In particular, they describe more precisely the main and

alternative sequences of the use case, depicting exactly how they diverge from each other.

22.2.1 Use Case Description for Browse Catalog

Use case name: Browse Catalog

Summary: Customer browses World Wide Web catalog, views various catalog items from the supplier's catalog, and selects items from the catalog.

Actor: Customer

Precondition: Customer browser is linked to supplier catalog Web site.

Main sequence:

1. Customer requests to browse catalog.
2. System displays catalog information to customer.
3. Customer selects items from catalog
4. System displays an itemized list containing each item description and price, as well as the total price.

Alternative sequence: Step 3: Customer does not select item and exits.

Postcondition: System has displayed list of selected catalog items.

The activity diagram describing the sequence of use case activities for Browse Catalog, which corresponds to the preceding description of the main sequence of the use case, is depicted in [Figure 22.2](#).

22.2.2 Use Case Description for Make Order Request

Use case name: Make Order Request

Summary: Customer enters an order request to purchase catalog items. The customer's credit card is checked for validity and sufficient credit to pay for the requested catalog items.

Actor: Customer

Precondition: Customer has selected one or more catalog items

Main sequence:

1. Customer provides order request and customer account Id to pay for purchase.
2. System retrieves customer account information, including the customer's credit card details.
3. System checks the customer's credit card for the purchase amount and, if approved, creates a credit card purchase authorization number.
4. System creates a delivery order containing order details, customer Id, and credit card authorization number.

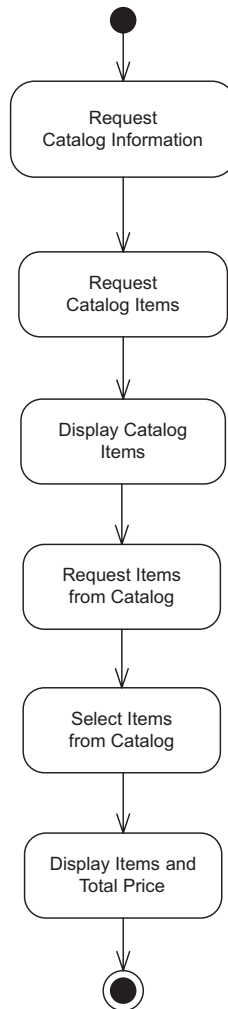


Figure 22.2. Activity diagram for Browse Catalog use case

5. System confirms approval of purchase and displays order information to customer.

Alternative sequences:

Step 2: If customer does not have an account, the system prompts the customer to provide information in order to create a new account. The customer can either enter the account information or cancel the order.

Step 3: If authorization of the customer's credit card is denied (e.g., invalid credit card or insufficient funds in the customer's credit card account), the system prompts the customer to enter a different credit card number. The customer can either enter a different credit card number or cancel the order.

Postcondition: System has created a delivery order for the customer.

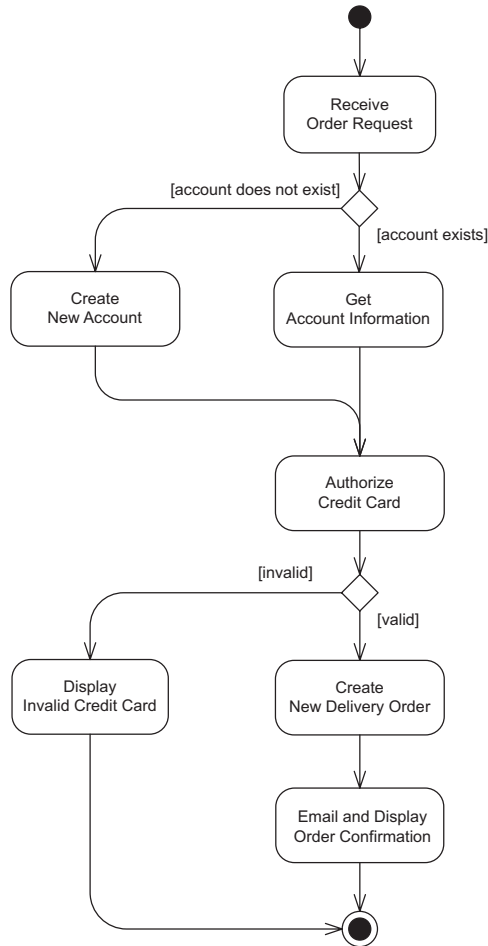


Figure 22.3. Activity diagram for Make Order Request use case

The activity diagram for Make Order Request (Figure 22.3) depicts the activities that correspond to the main sequence of the use case, for making an order and authorizing the credit card, as well as the two alternatives that deal with creating a new account and denying credit card authorization.

22.2.3 Use Case Description for Process Delivery Order

Summary: Supplier requests a delivery order; system determines that the inventory is available to fulfill the order, and displays the order.

Actor: Supplier

Precondition: Supplier needs to process a delivery order and a delivery order exists.

Main sequence:

1. Supplier requests next delivery order.
2. System retrieves and displays delivery order.
3. Supplier requests inventory check on items for delivery order.
4. System determines that items are available in inventory to satisfy delivery order and reserves items.
5. System displays inventory information to Supplier and confirms that items are reserved.

Alternative sequence: Step 4: If item is out of stock, system displays warning message.

Postcondition: System has reserved inventory items for delivery order.

The activity diagram for Process Delivery Order (Figure 22.4) depicts the activities corresponding to the main sequence of the use case, for viewing and reserving inventory items, as well the alternative sequence dealing with inventory item(s) out of stock.

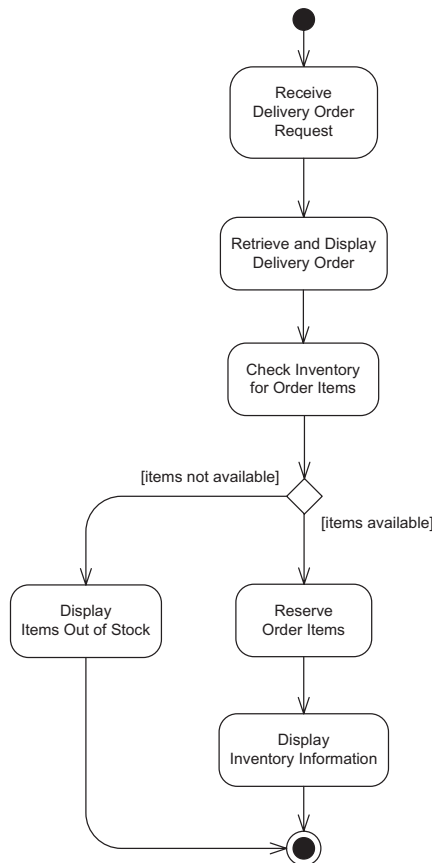


Figure 22.4. Activity diagram for Process Delivery Order use case

22.2.4 Use Case Description for Confirm Shipment and Bill Customer

Summary: Supplier prepares the shipment manually and then confirms that the delivery order is ready for shipment. System notifies customer that order is being shipped. System charges customer's credit card for purchase and commits inventory items removal.

Actor: Supplier

Precondition: Inventory items have been reserved for customer delivery order.

Main sequence:

1. Supplier prepares shipment manually and identifies that delivery order is ready to ship.
2. System retrieves customer account information, including the invoice and customer's credit card details.
3. System updates inventory to confirm purchase.
4. System charges customer's credit card for purchase and creates a credit card charge confirmation number.
5. System updates delivery order information with credit card charge confirmation.
6. System sends email confirmation to customer.
7. System displays confirmation to the supplier to complete shipment of the delivery order.

Postcondition: System has committed inventory, charged customer, and sent confirmation.

The activity diagram for Confirm Shipment and Bill Customer ([Figure 22.5](#)) depicts the activities corresponding to the main sequence of the use case.

22.2.5 Activity Diagram for View Order Use Case

In this simple use case, the customer requests to view an order. The activity diagram for the View Order use case is depicted in [Figure 22.6](#).

22.3 STATIC MODELING

This section describes the static model, which consists of the system context model and the entity class model. This section also discusses the use of brokering technology in online shopping service-oriented architectures.

22.3.1 Software System Context Modeling

The software system context model depicts two external user classes that are depicted as actors: the Customer and Supplier classes. The context diagram ([Figure 22.7](#))

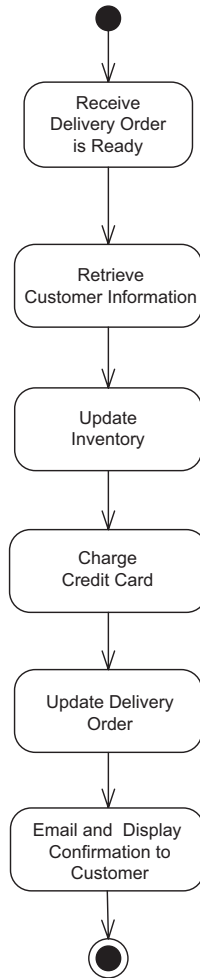


Figure 22.5. Activity diagram for Confirm Shipment and Bill Customer use case

is very similar to the use case diagram because the external classes correspond to the actors on the use case diagram.

22.3.2 Static Entity Class Modeling of the Problem Domain

A static model of the problem domain is developed and depicted on a class diagram (Figure 22.8). Because this is a data-intensive application, the emphasis is on the entity classes. The static entity class model shows the entity classes and the relationships among these classes. The classes include customer classes (Customer and Customer Account), supplier classes (Supplier, Inventory, and Catalog), and classes that deal with the customer's order such as Delivery Order, which is an aggregation of Item. The attributes for the classes are shown in Figure 22.9. This example is also described in more detail in Chapter 7.

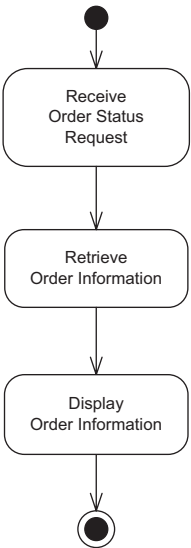


Figure 22.6. Activity diagram for View Order use case

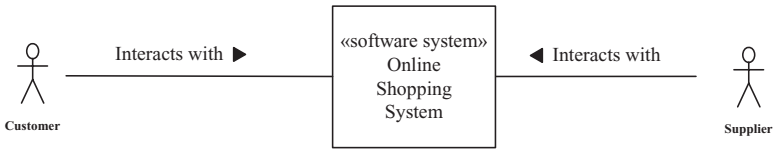


Figure 22.7. Online Shopping System Software System Context class diagram

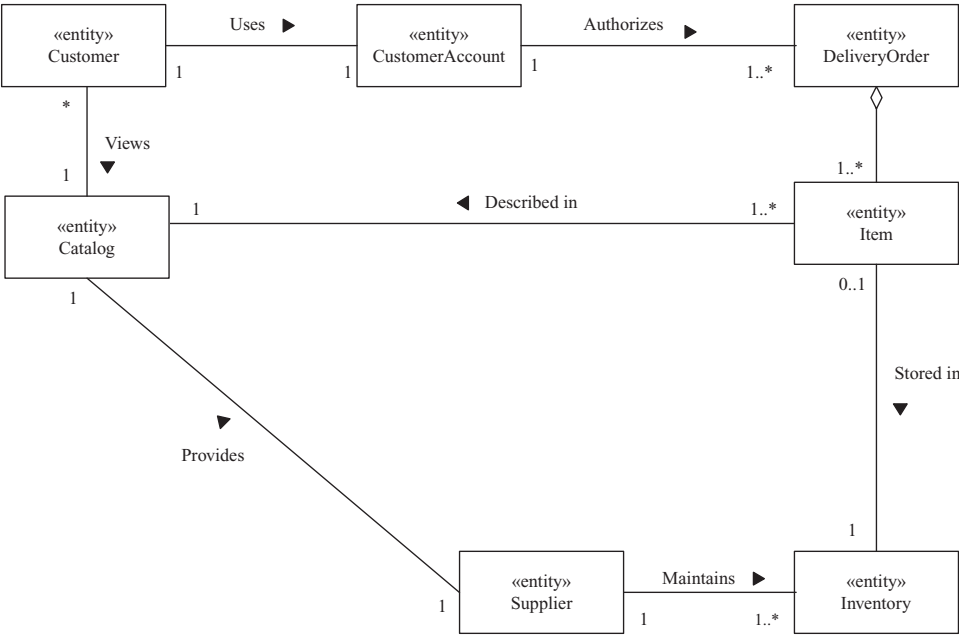


Figure 22.8. Conceptual static model for Online Shopping System entity classes

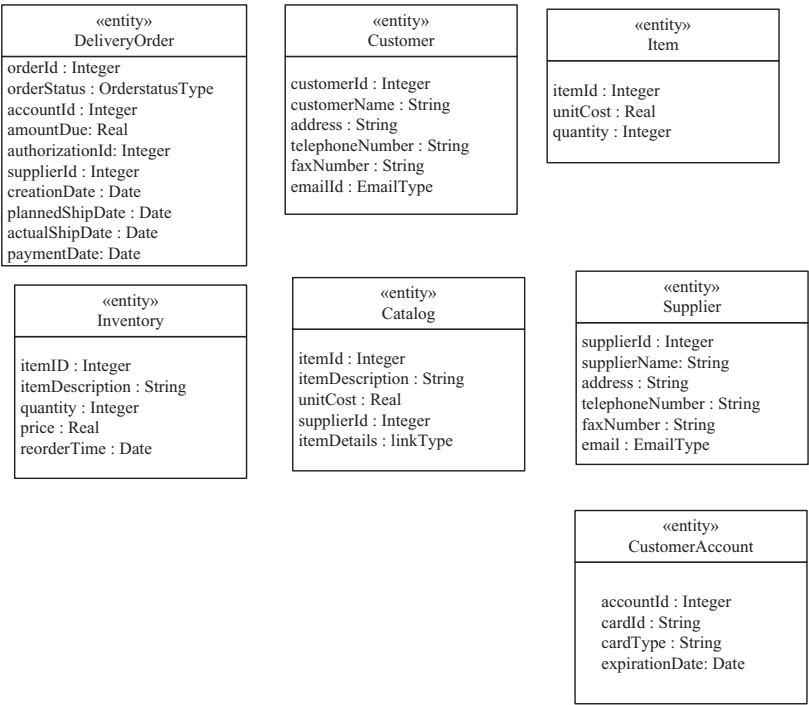


Figure 22.9. Entity classes for the Online Shopping System

22.4 OBJECT AND CLASS STRUCTURING

The entity classes determined in the previous section are integrated into a service-oriented architecture by means of service classes. Catalog Service, Customer Account Service, Delivery Order Service, and Inventory Service are service classes that provide access to the entity classes (Figure 22.10). Catalog Service uses the Catalog and Supplier entity classes. Customer Account Service uses the Customer

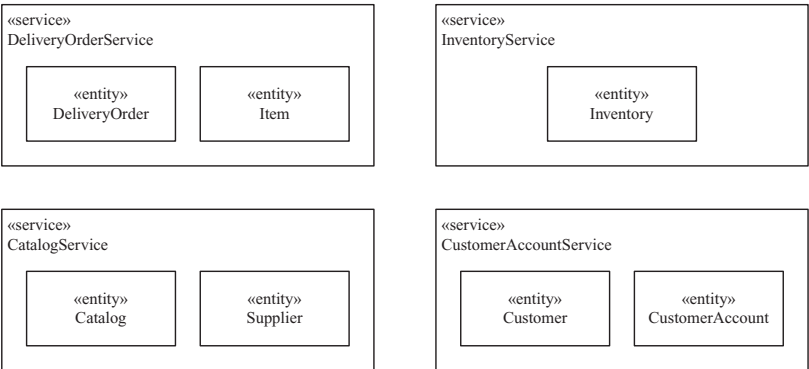


Figure 22.10. Service and entity classes for the Online Shopping System

Account and Customer entity classes. Delivery Order Service uses the Delivery Order and Item entity classes. The Inventory Service uses the Inventory entity class.

There is also a service class, Credit Card Service, which deals with credit card authorization and charging. In this way, the credit card billing is integrated with the customer purchasing and supplier delivery. Another service class is Email Service, which enables the Online Shopping System to send email messages to customers.

User interaction classes are needed to interact with the external users – in particular, Customer Interaction and Supplier Interaction, which correspond to the actors in the use cases. In addition, to coordinate and sequence the customer and supplier access to the online shopping services, two coordinator objects, Customer Coordinator and Supplier Coordinator, are provided. A third autonomous coordinator, Billing Coordinator, is needed to deal with billing customers. The classes in the Online Shopping System are depicted in [Figure 22.11](#).

22.5 DYNAMIC MODELING

For each use case, a communication diagram is developed that depicts the objects that participate in the use case and the sequence of messages passed between them.

22.5.1 Dynamic Modeling for Browse Catalog

In the communication diagram for the Browse Catalog use case ([Figure 22.12](#)), Customer Interaction interacts with Customer Coordinator, which in turn communicates with Catalog Service. The message descriptions are as follows:

-
- B1:** The customer makes a catalog request via Customer Interaction.
 - B2:** Customer Coordinator is instantiated to assist the customer. On the basis of the customer's request, Customer Coordinator selects a catalog for the customer to browse.
 - B3:** Customer Coordinator requests information from Catalog Service.
 - B4:** Catalog Service sends catalog information to Customer Coordinator.
 - B5:** Customer Coordinator forwards the information to Customer Interaction.
 - B6:** Customer Interaction displays the catalog information to the customer.
 - B7:** The customer makes a catalog selection through Customer Interaction.
 - B8:** Customer Interaction passes the request on to Customer Coordinator.
 - B9:** Customer Coordinator requests the catalog selection from Catalog Service.
 - B10:** Catalog Service confirms the availability of the catalog items and sends the item prices to Customer Coordinator.
 - B11:** Customer Coordinator forwards the information to Customer Interaction.
 - B12:** Customer Interaction displays the catalog information to the customer, including item prices and total.
-

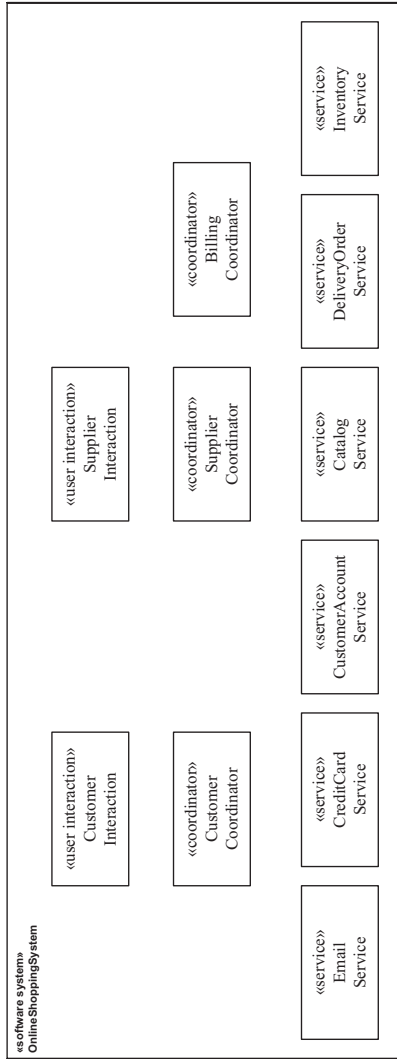


Figure 22.11. Class structuring for the Online Shopping System

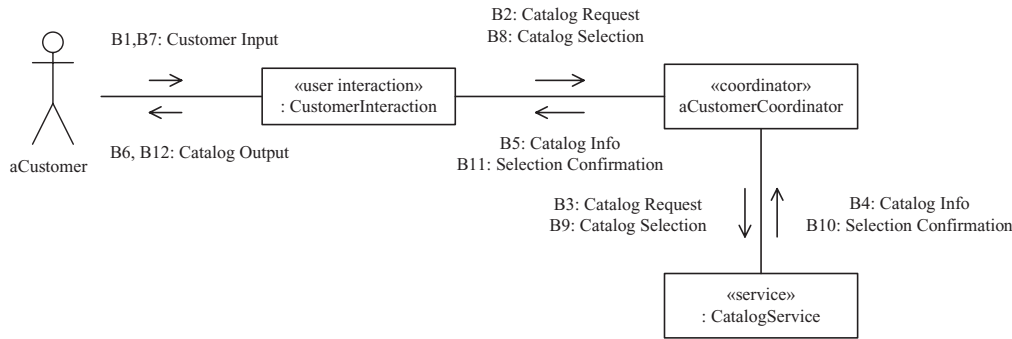


Figure 22.12. Communication diagram for the Browse Catalog use case

22.5.2 Dynamic Modeling for Make Order Request

In the communication diagram for the Make Order Request use case (Figure 22.13), a customer provides the account information, which is used to access the Customer Account Service. Credit card details are sent by Customer Coordinator to the Credit Card Service for approval. The Customer Coordinator then sends a new order request to Delivery Order Service and a confirmation email to the Email Service. The message descriptions are as follows:

- M1:** The customer provides order request to Customer Interaction.
- M2:** Customer Interaction sends the order request to Customer Coordinator.
- M3, M4:** Customer Coordinator sends the account request to Customer Account Service and receives the account information, including the customer’s credit card details.

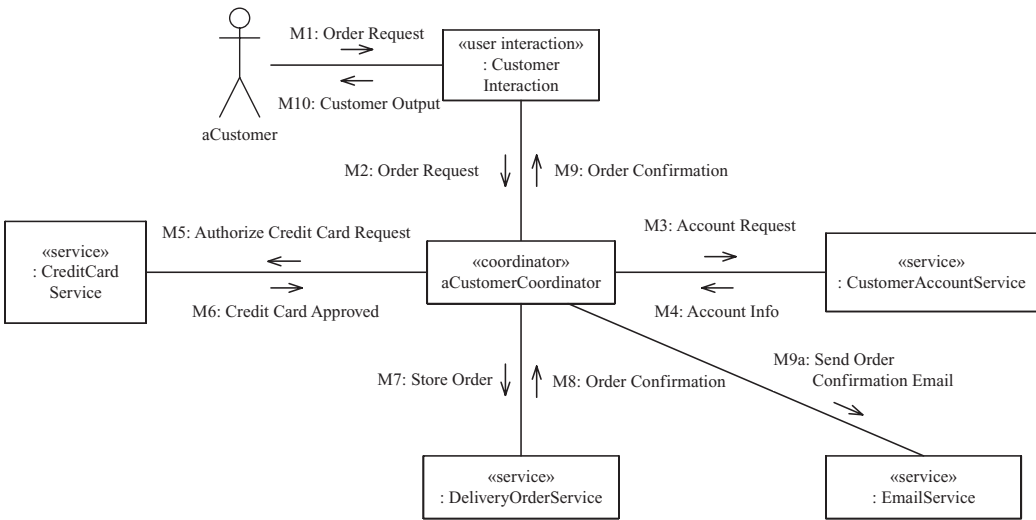


Figure 22.13. Communication diagram for the Make Order Request use case

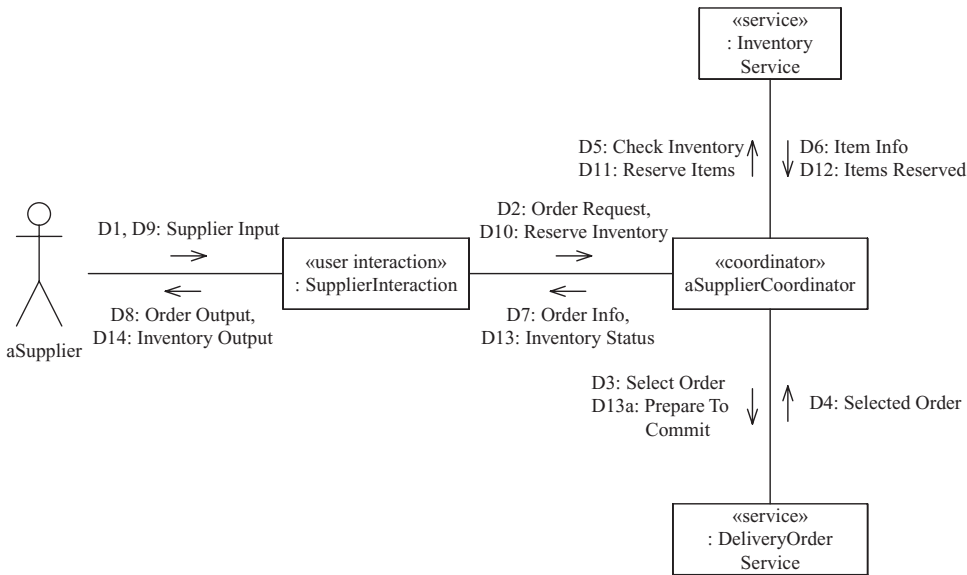


Figure 22.14. Communication diagram for the Process Delivery Order use case

M5: Customer Coordinator sends the customer's credit card information and charge authorization request to Credit Card Service (this is equivalent to a Prepare to Commit message).

M6: Credit Card Service sends a credit card approval to Customer Coordinator (this is equivalent to a Ready to Commit message).

M7, M8: Customer Coordinator sends order request to Delivery Order Service.

M9, M9a: Customer Coordinator sends the order confirmation to Customer Interaction and sends an email of the order confirmation to the customer via the Email Service.

M10: Customer Interaction outputs the order confirmation to the customer

Alternative scenarios for this use case are that the customer does not have an account, in which case a new account will be created, or that the credit card authorization is denied, in which case the customer has the option of selecting a different card. These alternative scenarios are described in Chapter 9.

22.5.3 Dynamic Modeling for Process Delivery Order

In the communication diagram for the next use case, Process Delivery Order (Figure 22.14), Supplier Coordinator requests Delivery Order Service for a new delivery order, and Delivery Order Service selects a delivery order. Supplier Coordinator requests Inventory Service to check the inventory and sends the order and inventory information to the supplier via the user interaction object. The message descriptions are as follows:

D1: The supplier requests a new delivery order.

D2: Supplier Interaction sends the supplier request to Supplier Coordinator.

- D3:** Supplier Coordinator requests Delivery Order Service to select a delivery order.
 - D4:** Delivery Order Service sends the delivery order to Supplier Coordinator.
 - D5:** Supplier Coordinator requests inventory item check.
 - D6:** Inventory Service returns item information.
 - D7:** Supplier Coordinator sends order status to Supplier Interaction.
 - D8:** Supplier Interaction displays the delivery order information to the supplier.
 - D9:** Supplier requests the system to reserve the items in inventory.
 - D10:** Supplier Interaction sends the supplier request to reserve inventory to Supplier Coordinator.
 - D11:** Supplier Coordinator requests Inventory Service to reserve the items in inventory (this is equivalent to a Prepare to Commit message)
 - D12:** Inventory Service confirms reservation of items to Supplier Coordinator (this is equivalent to a Ready to Commit message).
 - D13:** Supplier Coordinator sends the inventory status to Supplier Interaction.
 - D14:** Supplier Interaction displays the inventory information to the supplier.
-

An alternative scenario for this use case (not shown in figure) is that the item is out of stock, in which case Inventory Service returns an Out of Stock message, which is then displayed to the supplier.

22.5.4 Dynamic Modeling for Confirm Shipment and Bill Customer

In the communication diagram for the Confirm Shipment and Bill Customer use case (Figure 22.15), the supplier prepares the shipment manually. The supplier then sends the Ready for Shipment message to the Supplier Coordinator, which requests Inventory Service to commit the inventory and send a Ready for Shipment message to Billing Coordinator. The Billing Coordinator retrieves the invoice from the Delivery Order Service, account information from Customer Account Service, and charges the customer through Credit Card Service. The updates to the credit card, delivery order, and inventory are coordinated using the two-phase commit protocol (see Chapter 16). The message descriptions are as follows:

-
- S1:** The supplier inputs the shipping information.
 - S2:** Supplier Interaction sends the Ready for Shipment request to Supplier Coordinator.
 - S3:** Supplier Coordinator sends the Order Ready for Shipment message to Billing Coordinator.
 - S4:** Billing Coordinator sends Prepare to Commit order to Delivery Order Service.
 - S5:** Delivery Order Service replies with Ready to Commit message and invoice containing order Id, account Id, and amount.
 - S6, S7:** Billing Coordinator sends account request to Customer Account Service, which responds with account information.

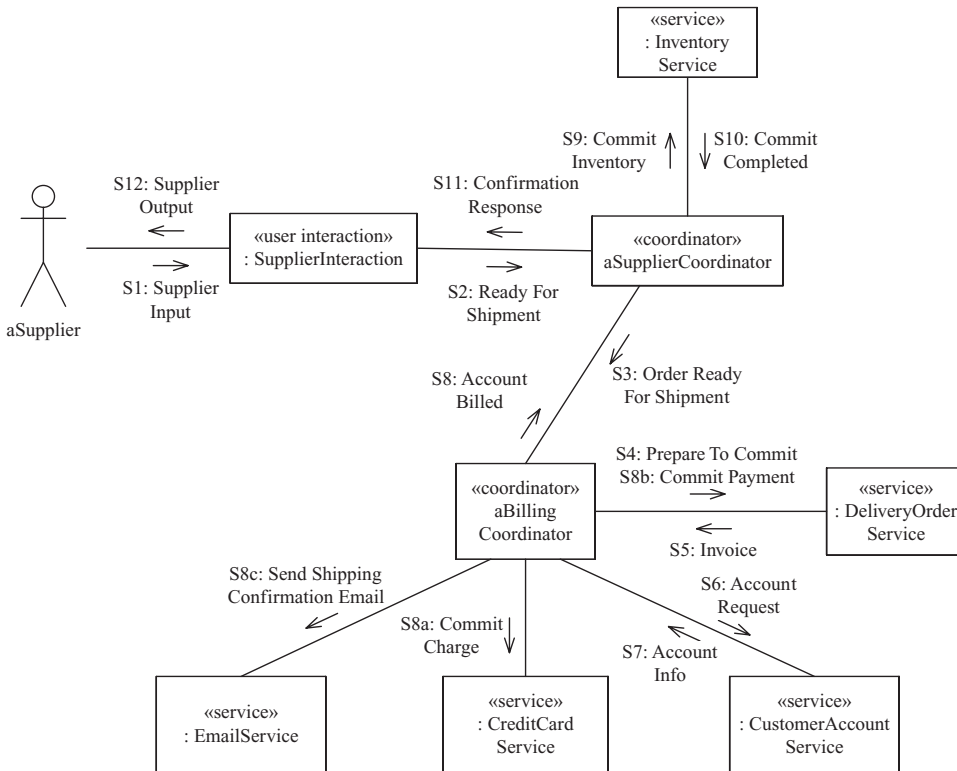


Figure 22.15. Communication diagram for the Confirm Shipment and Bill Customer use case

S8, S8a, S8b, S8c: Billing Coordinator sends Commit Charge message to Credit Card Service, Commit Payment message to Delivery Order Service, confirmation email to customer through Email Service, and Account Billed message to Supplier Coordinator.

S9, S10: Supplier Coordinator sends Commit Inventory message to Inventory Service, which responds that commit is completed.

S11, S12: Supplier Coordinator sends confirmation response to Supplier Interaction, which in turn sends the shipping confirmation message to the supplier.

22.5.5 Dynamic Modeling for View Order

In the communication diagram for the View Order use case (Figure 22.16), Customer Interaction interacts with Customer Coordinator, which in turn communicates with Delivery Order Service. The message descriptions are as follows:

V1, V2: The customer makes an order invoice request via Customer Interaction.

V3: Customer Coordinator makes an order request to Delivery Order Service.

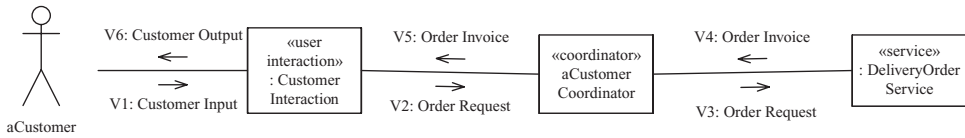


Figure 22.16. Communication diagram for the View Order use case

- V4:** Delivery Order Service sends order invoice information to Customer Coordinator.
- V5:** Customer Coordinator forwards the information to Customer Interaction.
- V6:** Customer Interaction displays the order information to the customer.

22.6 BROKER AND WRAPPER TECHNOLOGY SUPPORT FOR SERVICE-ORIENTED ARCHITECTURE

Several legacy databases are used in the Online Shopping System. Many of the entity classes in the static model represent persistent data stored in legacy databases. Each legacy database is a stand-alone database that resides on a mainframe. These databases need to be integrated into the application, by means of a broker and wrapper technology. Chapter 15 provides information on database wrapper classes, and Chapter 16 provides information on Broker patterns.

Although different legacy databases exist, object broker and wrapper technology provide a systematic way of integrating the disparate legacy databases into a service-oriented architecture. The legacy databases in the supplier organization are the catalog database, the inventory database, the customer account service, and the delivery order database.

Database wrapper classes are designed to provide an object-oriented interface to the legacy databases that hides the details of how to read and update the individual databases. To integrate these databases into the online shopping application, service classes are designed that access the legacy databases through database wrapper classes.

22.7 DESIGN MODELING

This section describes the architectural structure and communication patterns used in the design, the concurrent software design, service and component interface design, and the design of the service-oriented architecture, which fully integrates services and components.

22.7.1 Overview of Service-Oriented Architecture

In service-oriented architectures, services register their service name and location with a broker. Clients can then discover new services by using the Service Discovery pattern (also known as yellow pages) to query the broker for services of a given

type. The client can then choose a service and make a white pages request to the broker.

The Online Shopping System is designed as a layered architecture based on the Layers of Abstraction architecture pattern. The software architecture consists of three layers – a service layer, a coordinator layer, and a user interaction layer. Furthermore, because this system needs to be highly flexible and distributed, the decision is made to design a service-oriented architecture, in which distributed components can discover services and communicate with them.

Each component is depicted with the component stereotype (what kind of component it is, as specified by the component structuring criteria). The design of the component and service interfaces are determined by analysis of the communication diagrams for each use case.

22.7.2 Layered Software Architecture

The components are structured into the layered architecture such that each component is in a layer where it depends on components in the layers below but not the layers above. This layered architecture is based on the Layers of Abstraction pattern (Chapter 12). The layered architecture facilitates future adaptation of the online shopping software architecture. User interaction components at the user interaction layer communicate only with coordinator components. Coordinator components communicate with services. Applying the component structuring criteria, the following components and services, organized by layer, are determined, as depicted in Figure 22.17:

Layer 1: Service Layer. There are six services ; four are part of the application, and two are external services. The application services are Catalog Service, Delivery Order Service, Inventory Service, and Customer Account Service. The external services are Credit Card Service (one for each credit card company, such as Mastercard and Visa), which is used for charging customer purchases, and Email Service, for sending email messages to customers.

Layer 2: Coordination Layer. There are three coordinator components: Supplier Coordinator, Customer Coordinator, and Billing Coordinator.

Layer 3: User Layer. There are two user interaction components: Supplier Interaction and Customer Interaction.

22.7.3 Architectural Communication Patterns

To handle the variety of communication between the components in the software architecture, several communication patterns are applied:

- **Synchronous Message Communication with Reply.** This is the typical service-oriented architecture pattern of communication and is used when the client needs information from the service and cannot proceed before receiving the response. This pattern is used between user interaction clients and coordinators. It is also used between coordinators and various services.

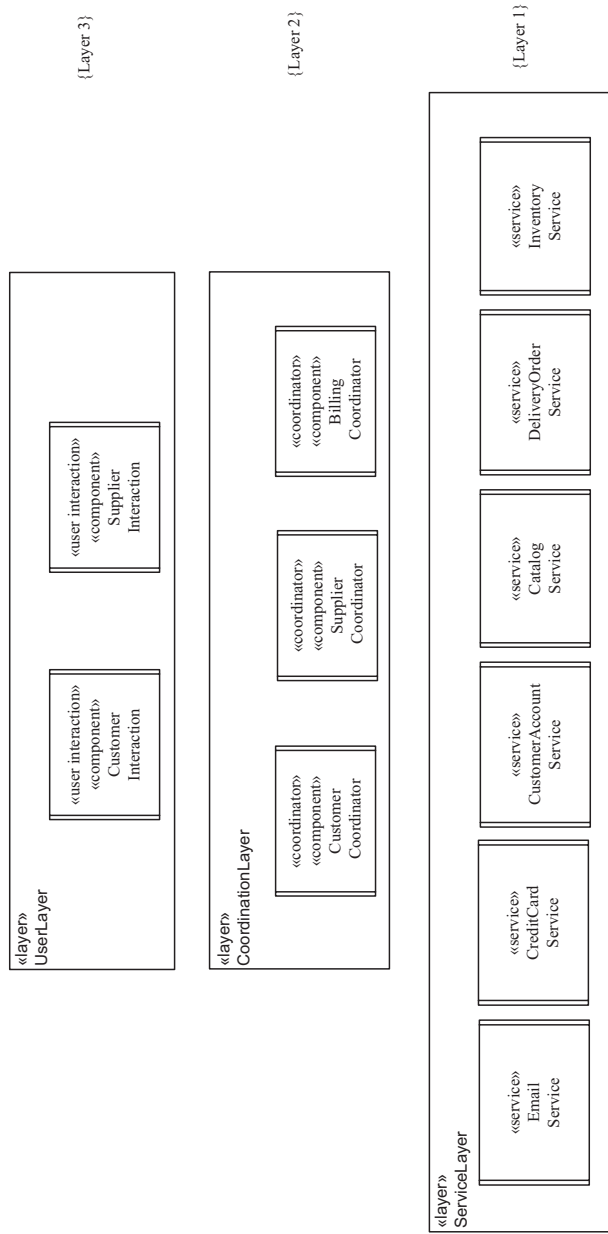


Figure 22.17. Layered architecture for Online Shopping System

- **Broker Handle.** Each service registers service information, including service name, service description and location with the broker. The Broker Handle pattern allows clients to query the broker to determine the services to which they should be connected.
- **Service Discovery.** Service Discovery patterns are used by service requesters to discover new services. They could be used to discover new catalogs to browse.
- **Bidirectional Asynchronous Message Communication.** This pattern is used for communication between the Supplier Coordinator and Billing Coordinator to asynchronously communicate with each other in both directions.
- **Two-Phase Commit.** This pattern is used to ensure that updates to inventory, credit card, and delivery order are atomic, so either all updates are committed or all are aborted.

22.7.4 Concurrent Software Design

The service-oriented architecture is designed by integrating the use case-based interaction diagrams described in Section 22.5, and then designing the message interfaces. The concurrent software design is depicted on the concurrent communication diagram for the Online Shopping System (Figure 22.18), which depicts the concurrent components and services. It represents the integrated communication diagram determined from the individual communication diagrams supporting the use cases. Furthermore, the design of the message interactions is also depicted.

To keep the design simple, the Synchronous Message Communication with Reply pattern has been widely used in this case study. As described in Chapter 12, however, this approach has the disadvantage of suspending the client while it awaits a response from the service. An alternative design to avoid suspending the client is to use the Asynchronous Message Communication with Callback pattern, as described in Chapter 15. The Bidirectional Asynchronous Communication pattern is used for Supplier Coordinator and Billing Coordinator to communicate with each other in both directions.

22.7.5 Service Interface Design

The service interfaces are designed as follows. Each service has one provided interface through which the service operations are accessed. Figure 22.19 depicts the service interfaces and ports. The clients of the service invoke the appropriate operations provided by the interface synchronously.

The service operations are designed by considering how each individual service is accessed on the use case-based interaction diagrams. Typically, each service is accessed in different ways corresponding to requests for different service operations. The interaction diagrams depict the messages arriving at the service (corresponding to service operation invocation and possible input parameters to the service) and the service response (corresponding to data returned by the service), which is either synchronous (as a synchronous message reply) or asynchronous (in a separate asynchronous message).

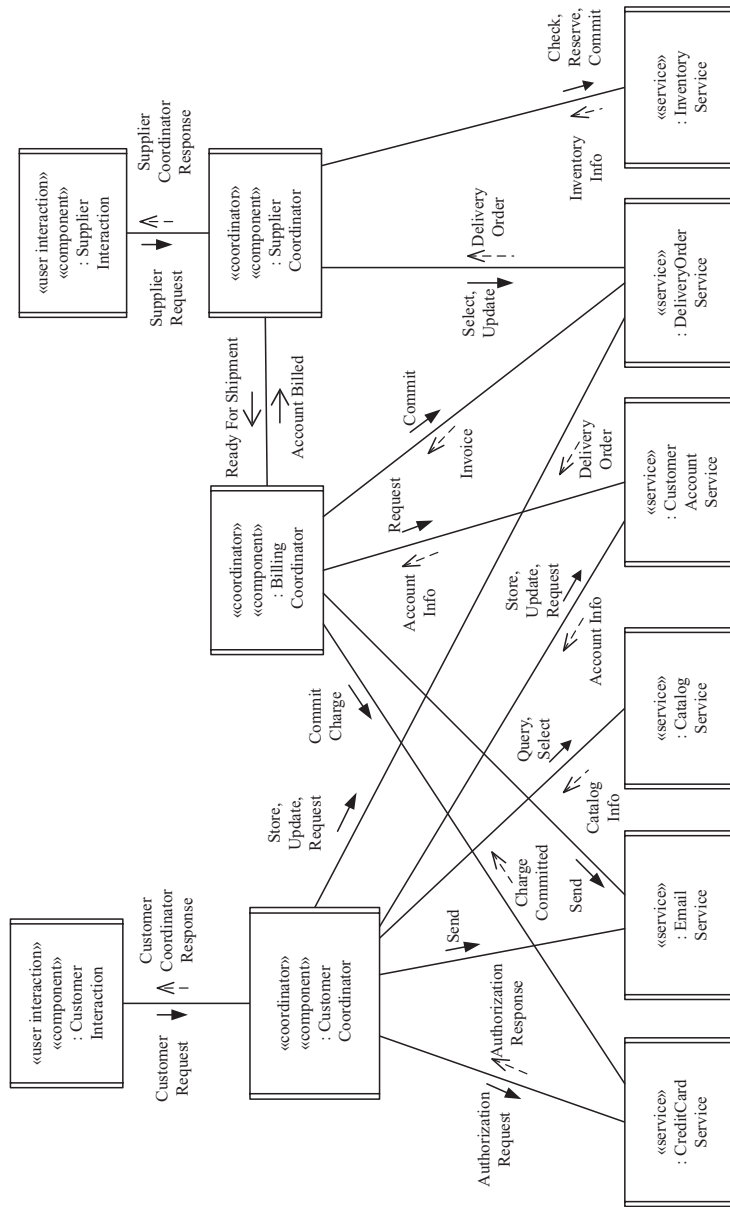


Figure 22.18. Concurrent communication diagram for Online Shopping System

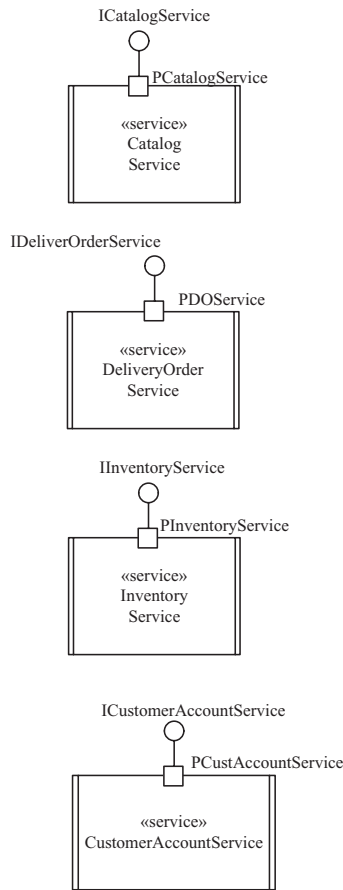


Figure 22.19. Component ports and interfaces for services

The Catalog Service has operations to request to view a catalog and to select catalog items (Figure 22.20). These operations access data maintained in the Catalog Info and Item Info entity classes. The access needs are determined from the Browse Catalog communication diagram (see Figure 22.7) and static model (see Figures 22.3 and 22.4). The requestCatalog operation returns catalog items of a given type and is determined from message B3 in Figure 22.12. The catalog information returned is given by the attributes of the Catalog entity class shown on Figures 22.9 and 22.20. The requestSelection operation is determined from message B9 in Figure 22.12, which returns (determined from message B10) the item information for the specific item.

The Customer Account Service has operations to create a new account, update the account, and read an account (Figure 22.21). These operations access data maintained in the Customer Account and Customer entity classes. The operations are determined from the Make Order Request communication diagram (see Figure 22.13). The requestAccount operation corresponds to message M3 in Figure 22.13 and message S6 in the Confirm Shipment and Bill Customer communication diagram (see Figure 22.15). The createAccount and updateAccount operations correspond

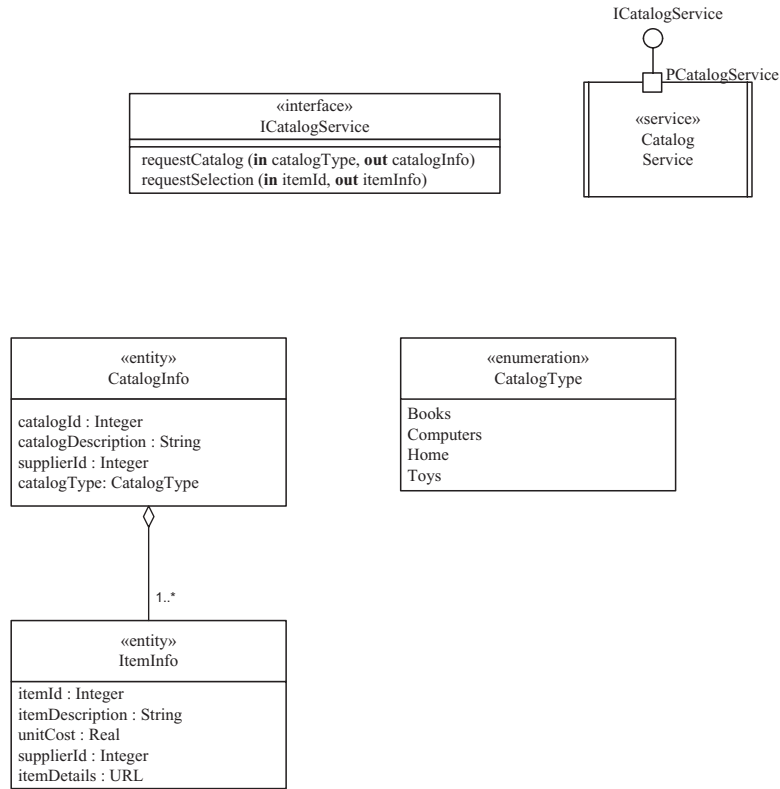


Figure 22.20. Service interface for Catalog Service

to alternatives to the main sequence of Make Order Request, as described in Section 22.5.2.

Confirm Shipment and Bill Customer (see Figure 22.8) also involves the Delivery Order Service (to commit the payment), the Credit Card Service (to authorize a charge), and the Email Service (to send a confirmation email).

The Make Order Request communication diagram also involves three other services: Delivery Order Service, Credit Card Service, and Email Service.

Delivery Order Service has several operations (Figure 22.22), which are determined as follows. The operations access data maintained in the Delivery Order and Item entity classes; the Invoice entity class contains data extracted from Delivery Order. Message M7 (on Figure 22.13) to store the delivery order corresponds to the store operation shown in Figure 22.22 for the Delivery Order Service. Message D3 to select the delivery order in the Process Delivery Order communication diagram (see Figure 22.14) corresponds to the select operation shown in Figure 22.22. Further operations are determined from the Confirm Shipment and Bill Customer (see Figure 22.15) communication diagram, particularly the Delivery Order Service messages to Prepare to Commit (message S4) the order and Commit the Payment (message S8b). The read operation is determined from message V3 on the View Order communication diagram (see Figure 22.16). The service interface for Delivery Order Service is depicted on Figure 22.22. The abort operation is invoked if the order is cancelled prior to shipment.

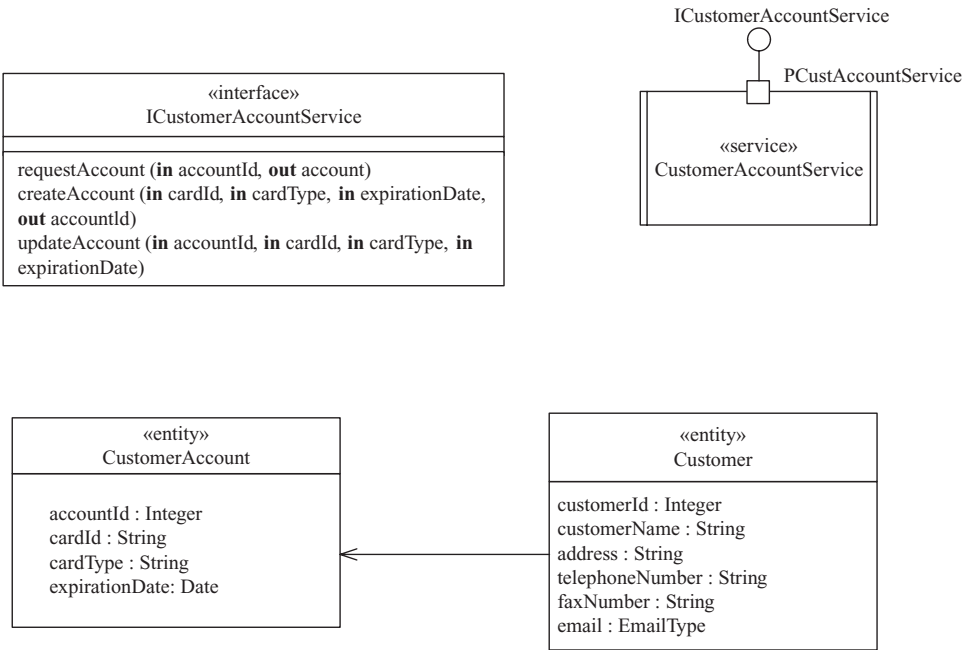


Figure 22.21. Service interface for Customer Account Service

Inventory Service needs operations to check inventory (determined from message D5 on Figure 22.14); reserve inventory (determined from message D11 on the same figure), which is equivalent to prepare to commit inventory; commit inventory (message S9 in Confirm Shipment and Bill Customer in Figure 22.15); and update

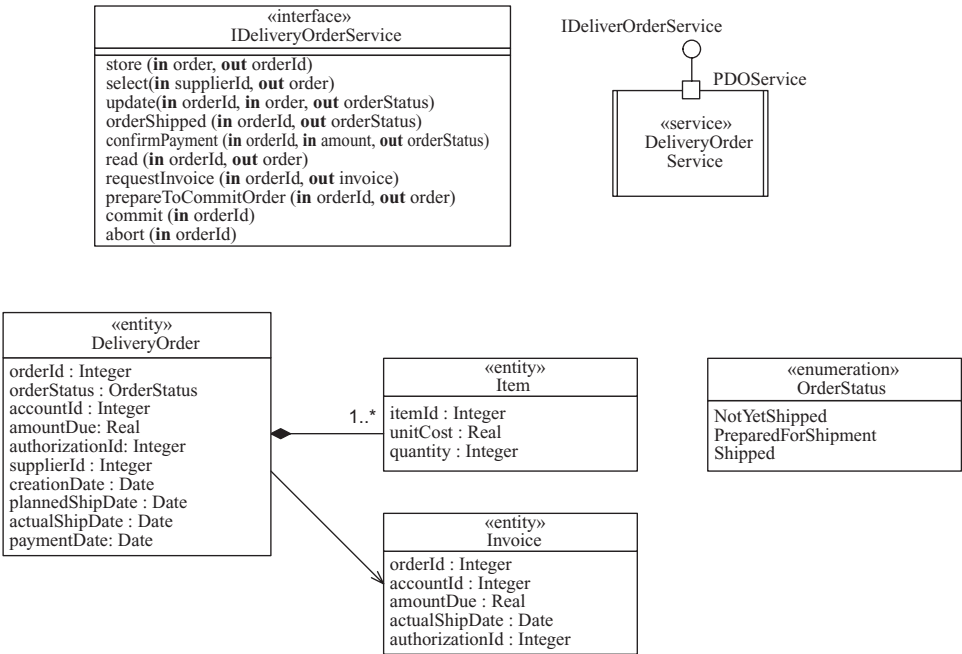


Figure 22.22. Service interface for Delivery Order Service

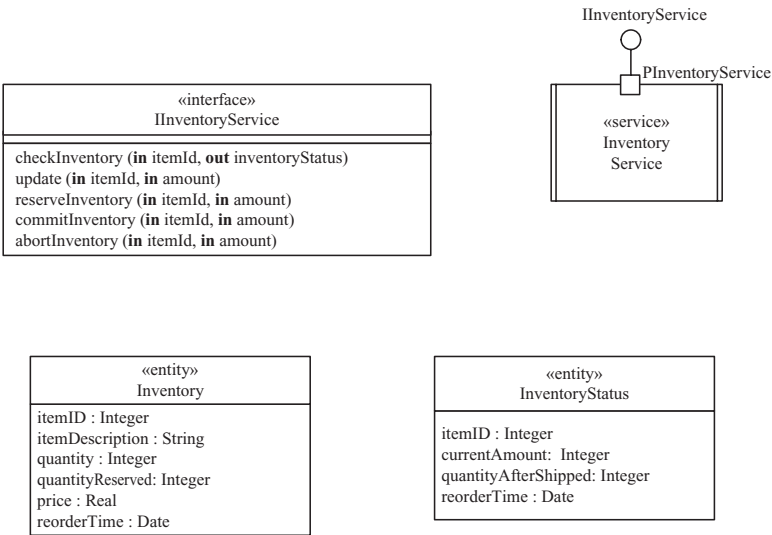


Figure 22.23. Service interface for Inventory Service

inventory. The abort operation is invoked if the order is cancelled and inventory released prior to shipment. The updateInventory service operation is needed when the inventory is replenished. This example is described in more detail in Chapter 16, Section 16.6.

Figure 22.15 shows the service interfaces for the external services: Credit Card Service and Email Service. Both external services have provided interfaces in the same way as the application services. The Credit Card Service supports one provided interface consisting of two operations – one for authorizing a credit card purchase (message M5 in Make Order Request) and the other for charging the credit card (message S8a in Confirm Shipment and Bill Customer). The Email Service has one provided interface with one operation to send an email message (message M9a in Make Order Request and message S8c in Confirm Shipment and Bill Customer).

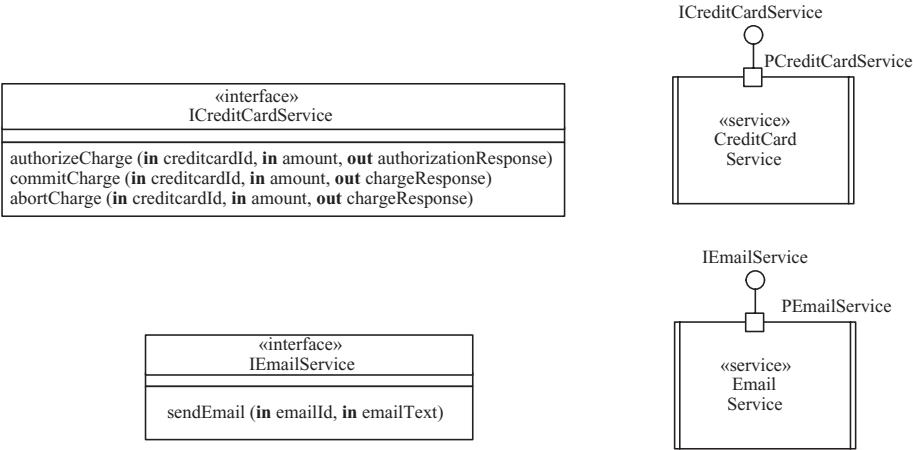


Figure 22.24. Service interfaces for Credit Card and Email services

22.7.6 Design of Service-Oriented Software Architecture

This section describes the design of the service-oriented architecture for the Online Shopping System, which is depicted on a composite structure diagram, as shown in [Figure 22.25](#). The service interface design has already been described in the previous sections; the user interaction and coordinator objects are designed as components and are described in the next section.

Each service has one port with one provided interface, whereas each coordinator component has one or more ports, with provided interfaces, required interfaces, or both. In the three-layer architecture, each client-user interaction component has one required port, which supports a required interface. Each service has one provided port, which supports a provided interface. The coordinators have ports with both required and provided interfaces because they act as intermediaries between the clients and services and need to communicate with several services. The provided interfaces of the services and the required ports of the coordinators are explicitly depicted in order to identify that services are intended to be discovered; thus, the binding between the service provider and the service requester is dynamic.

For customer requests, Customer Interaction is a client and thus has only a required port. Customer Coordinator has both required and provided ports. Customer Interaction only communicates with the Customer Coordinator, whereas the Customer Coordinator communicates with five services, of which there are two external services (Credit Card Service and Email Service) and three application services (Catalog Service, Delivery Order Service, and Customer Account Service).

For supplier requests, Supplier Interaction has a required port, and the Supplier Coordinator component has both provided and required interfaces. Supplier Interaction only communicates with the Supplier Coordinator, whereas the Supplier Coordinator communicates with both Delivery Order Service and Inventory Service. Supplier Coordinator also communicates with the Billing Coordinator. Billing Coordinator communicates with four services, of which there are two external services (Credit Card Service and Email Service) and two application services (Delivery Order Service and Customer Account Service). Because all customer payment is made by credit card, this necessitates the Credit Card Service, of which there could be many service instances, one for each credit card company. Each service instance is designed and implemented differently but must conform to the SOA credit card interface.

22.7.7 Design of Component Ports and Interfaces

The ports and interfaces for the user interaction and coordination components are described next. The Customer Interaction user interaction component has a required port, which consists of a required interface, as does Supplier Interaction, as shown in [Figures 22.25](#) and [22.26](#),

The component ports and interfaces for Customer Coordinator are depicted in [Figure 22.27](#), which also depicts the component's provided and required interfaces. Customer Coordinator has five required ports and one provided port. The required ports support required interfaces to Catalog Service, Customer Account Service, Delivery Order Service, Email Service, and Credit Card Service. The required interfaces are

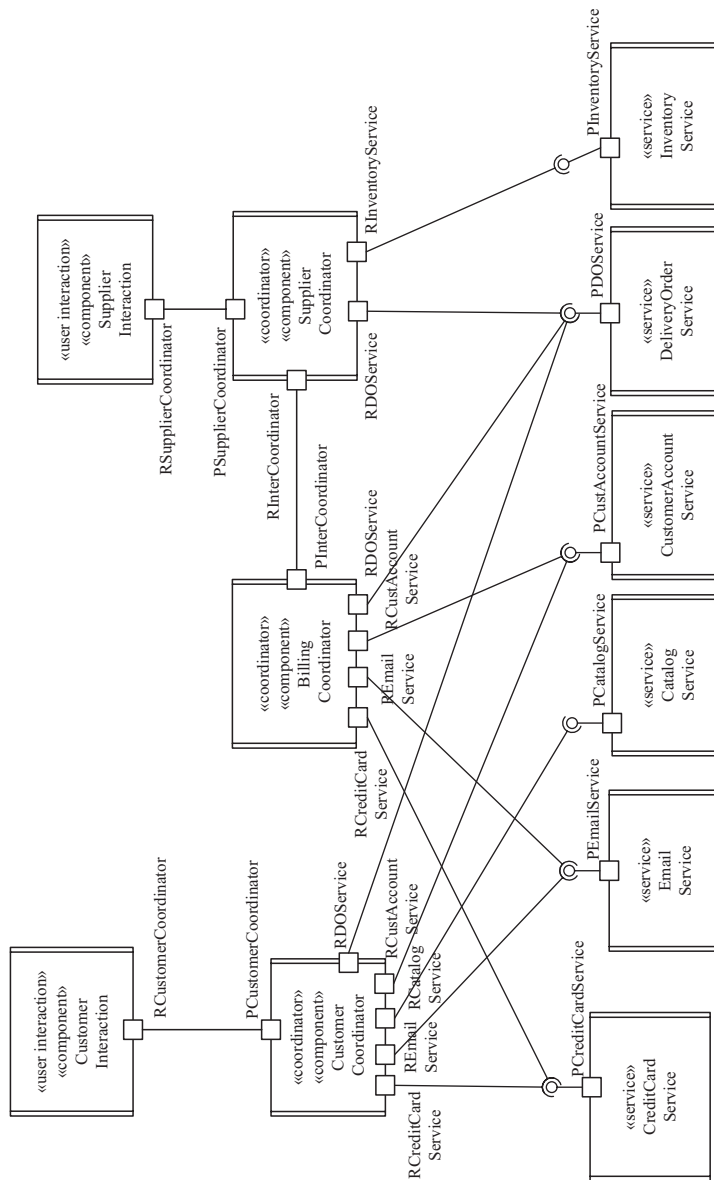


Figure 22.25. Service-oriented software architecture for the Online Shopping System

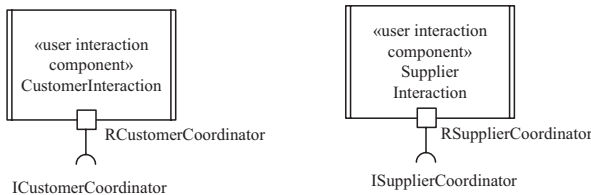


Figure 22.26. Component ports and interfaces for Customer Interaction and Supplier Interaction

because Customer Coordinator is a client when communicating with services. Each Customer Coordinator component has a provided port to communicate with Customer Interaction.

The component ports and interfaces for Supplier Coordinator are depicted in Figure 22.28. Supplier Coordinator receives supplier requests from Supplier Interaction via the PSupplierCoordinator port, which supports a provided interface called ISupplierCoordinator. Supplier Coordinator is a client of the Inventory Service and communicates with it via the required interface IInventoryService. It is also a client of and has the same required interface IDeliveryOrderService to Delivery Order Service as the Customer Coordinator. Supplier Coordinator also communicates with the Billing Coordinator, as depicted in Figure 22.15, sending it a message (S3) when an order is ready for shipment and billing. The interface for this communication is IShipment.

Billing Coordinator (see Figure 22.29) has required ports to communicate with two external services (Credit Card Service and Email Service) and two application services (Delivery Order Service and Customer Account Service). It also has a provided interface IShipment.

22.8 SERVICE REUSE

With the service-oriented architecture paradigm, once the services have been designed and their interfaces specified, the service interface information can be registered with a service broker. Services can be composed into new applications. This case study has described an Online Shopping System. However, other electronic commerce systems could be designed that would reuse the services provided by the

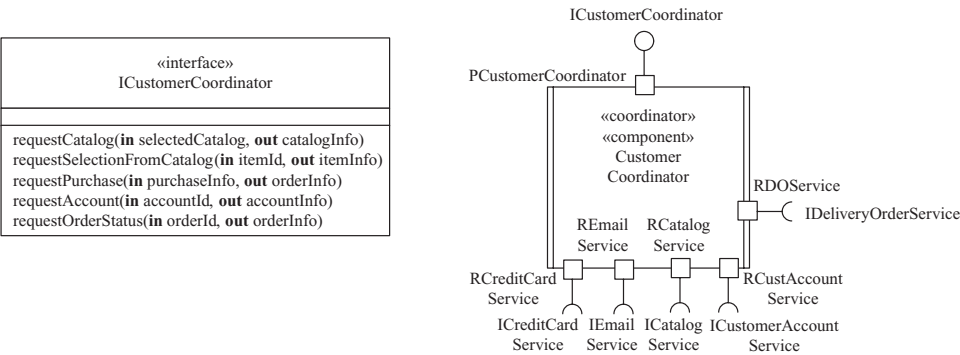


Figure 22.27. Component ports and interfaces for Customer Coordinator

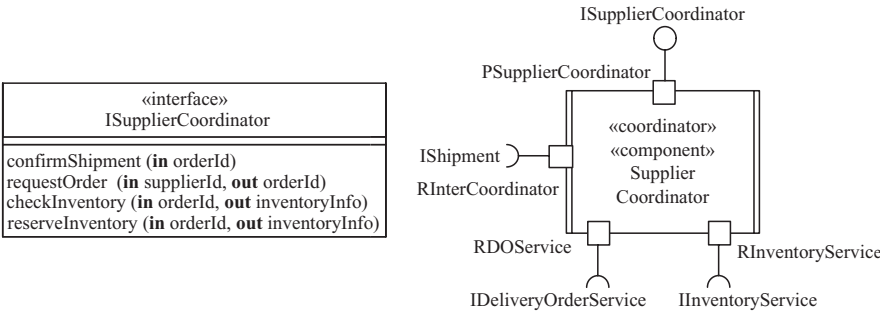


Figure 22.28. Component ports and interfaces for Supplier Coordinator

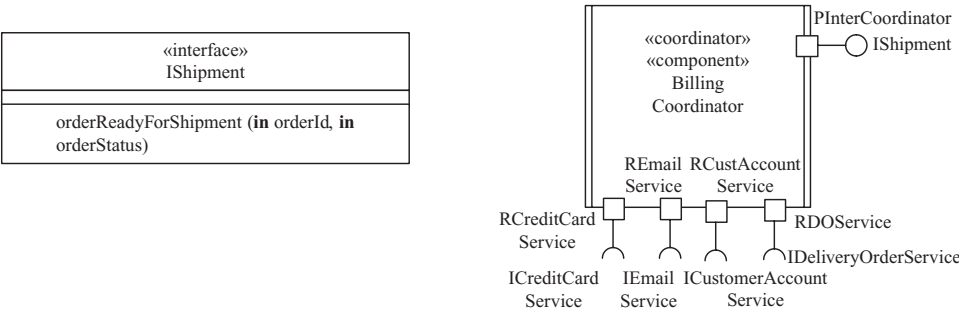


Figure 22.29. Component ports and interfaces for Billing Coordinator

Online Shopping System, such as Catalog Service, Delivery Order Service, and Inventory Service.

In a business to business (B2B) system, for example, instead of using customer accounts, contracts would be established between business customers and suppliers. Each contract would be between a specific business customer and a specific supplier, would be of a specified duration, and would have a maximum fiscal amount in a specified currency. A business customer would select items from the catalog, specify the contract that should be used, and then send the delivery order. Once the order shipment had been made, the payment to the supplier would be made through an electronic funds transfer from the business customer bank to the supplier bank.

The B2B system would necessitate the creation of additional services as well as different versions of the Customer and Supplier Coordinators. The Catalog, Delivery Order, and Inventory Services would be reused. However, new services would be required for Contract Service, Invoice Service, and Accounts Payable Service. A reusable service-oriented architecture for an Electronic Commerce software product line, consisting of kernel, optional, and variant components and services, is described in (Gomaa 2005a).